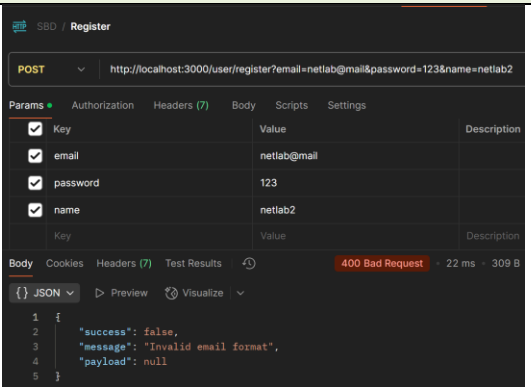
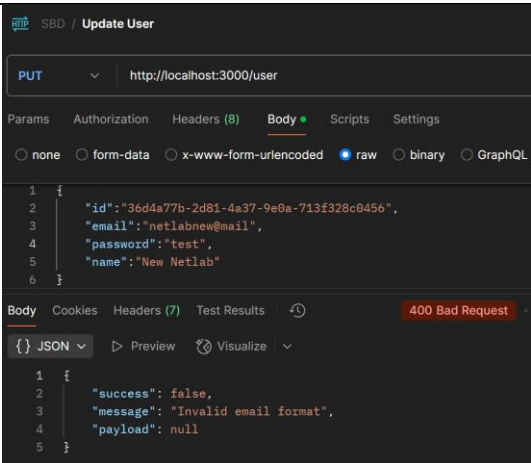


PRAKTIKUM SISTEM BASIS DATA

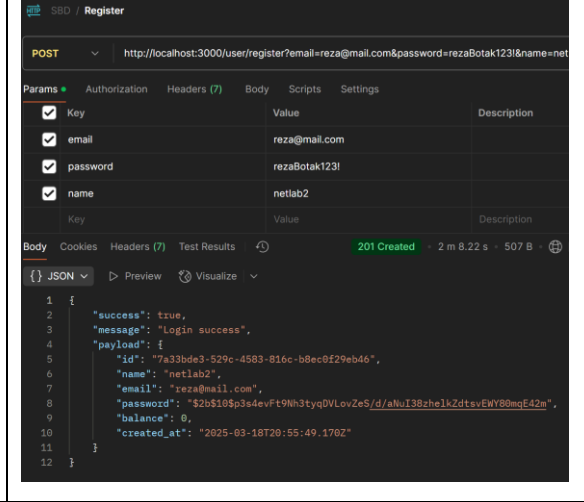
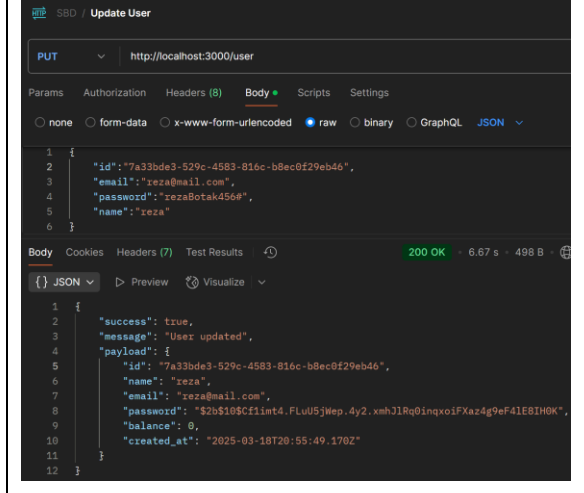
Nama	Muhammad Raditya Alif Nugroho	No. Modul	6
NPM	2306212745	Tipe	Case Study

1. Regex

Endpoint	Code	Response
/user/register	<pre>const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+\$/; const passRegex = /^(?=.*[0-9])(?=.*[!@#\$%^&*]).{8,}\$/; exports.register = async (req, res) => { const { email, password, name } = req.query; if (!email !password !name) { return baseResponse(res, false, 400, "Email, password, and name are required", null); } if (!emailRegex.test(email)) { return baseResponse(res, false, 400, "Invalid email format", null); } if (!passRegex.test(password)) { return baseResponse(res, false, 400, "Invalid Password", null); } }</pre>	
/user	<pre>exports.updateUser = async (req, res) => { const { id, email, password, name } = req.body; if (!id !email !password !name) { return baseResponse(res, false, 400, "ID, email, password, and name are required", null); } if (!emailRegex.test(email)) { return baseResponse(res, false, 400, "Invalid email format", null); } if (!passRegex.test(password)) { return baseResponse(res, false, 400, "Invalid Password", null); } }</pre>	

2. Hashing

Endpoint	Kode	Response
----------	------	----------

/user/register	<pre>const SALT_ROUNDS = 10; exports.register = async (user) => { try { const hashedPassword = await bcrypt.hash(user.password, SALT_ROUNDS); const res = await db.query("INSERT INTO users (name, email, password, balance) VALUES (\$1, \$2, \$3, \$4)", [user.name, user.email, hashedPassword, 0]); return res.rows[0]; } catch (error) { console.error("Error executing query:", error); } };</pre>	
/user	<pre>exports.updateUser = async (user) => { try { const hashedPassword = await bcrypt.hash(user.password, SALT_ROUNDS); const res = await db.query("UPDATE users SET name = \$1, email = \$2, password = \$3 WHERE id = \$4", [user.name, user.email, hashedPassword, user.id]); return res.rows[0]; } catch (error) { console.error("Error executing query:", error); } };</pre>	

3. Compare

```
exports.login = async (email, password) => {
  try {
    const res = await db.query("SELECT * FROM users WHERE email = $1", [email]);
    const user = res.rows[0];

    if (!user) return null;

    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) return null;
```

4. CORS

```

JS index.js  JS user.controller.js  JS user.repository.js
JS index.js > ...
1  const express = require('express');
2  const cors = require("cors");
3  require('dotenv').config();
4
5  const app = express();
6  const PORT = process.env.port || 3000;
7
8  const corsOptions = {
9    origin: "https://os.netlabdte.com",
10   methods: ["GET", "POST", "PUT", "DELETE"],
11 };
12 app.use(cors(corsOptions));

```

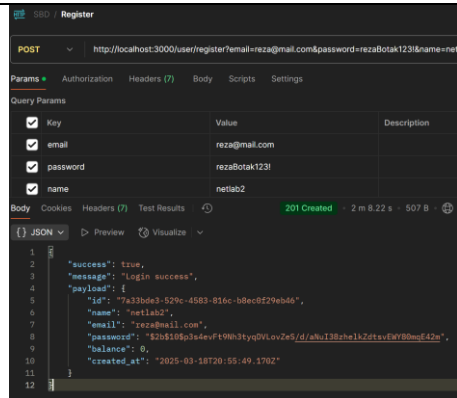
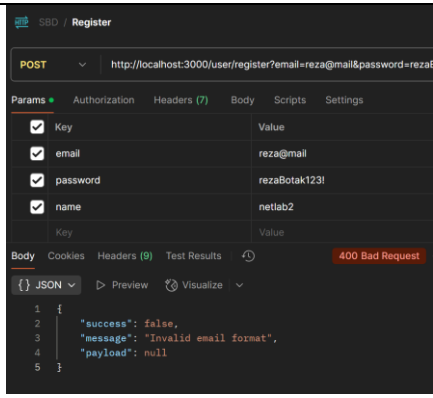
5. Query

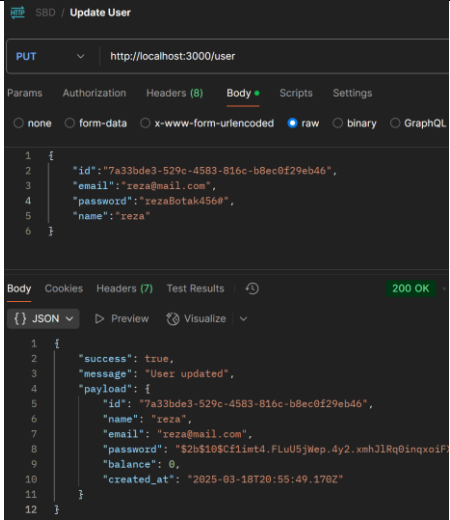
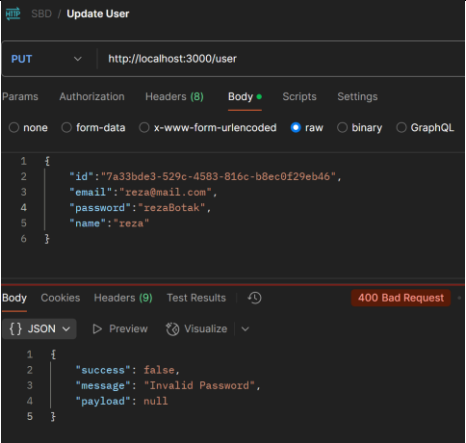
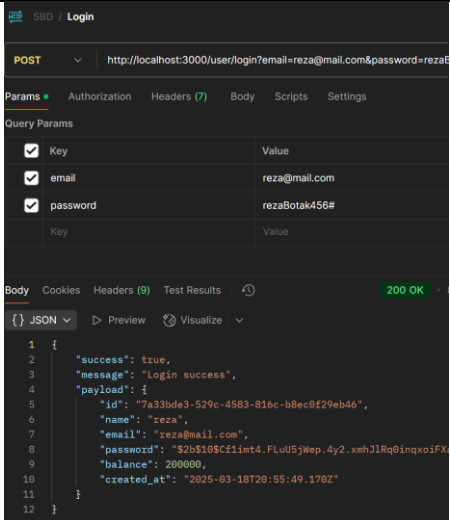
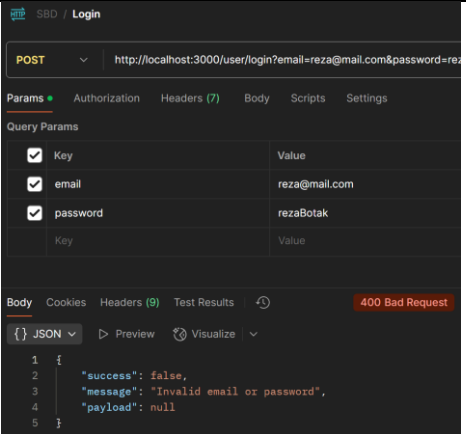
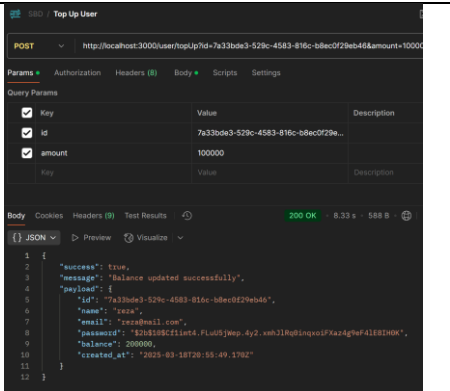
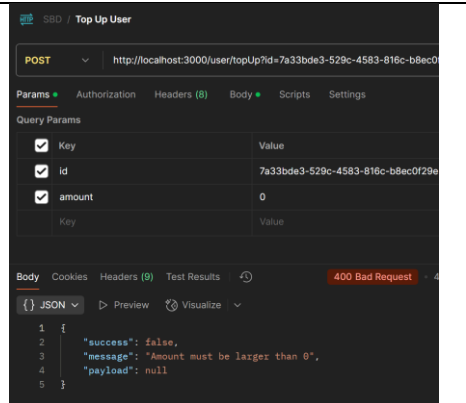
```

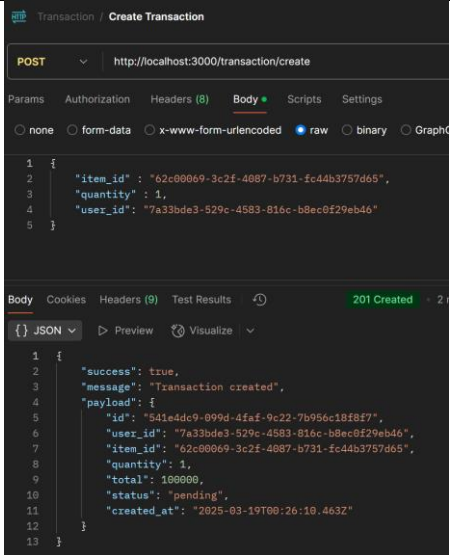
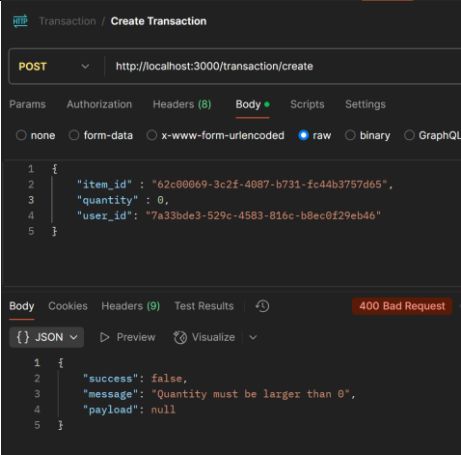
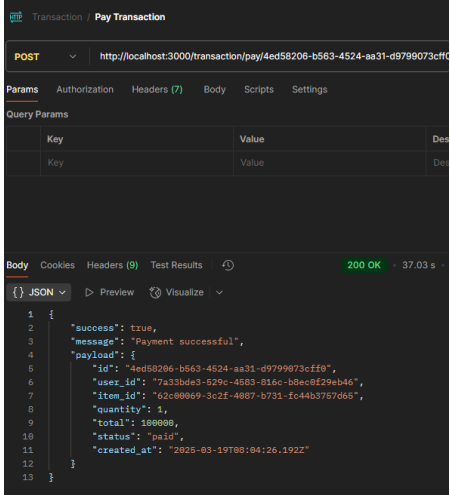
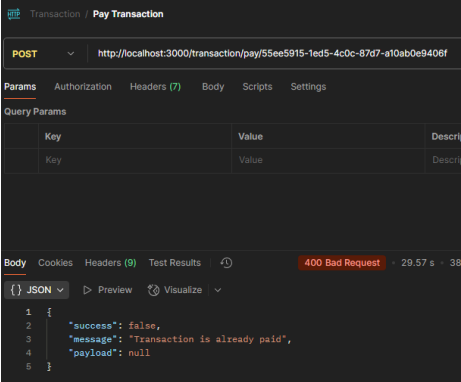
express_ditya=# CREATE TYPE transaction_status AS ENUM ('pending', 'paid');
CREATE TYPE
express_ditya=# CREATE TABLE IF NOT EXISTS transactions(
express_ditya(#   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
express_ditya(#   user_id UUID NOT NULL,
express_ditya(#   item_id UUID NOT NULL,
express_ditya(#   quantity INT NOT NULL,
express_ditya(#   total INT NOT NULL,
express_ditya(#   status transaction_status DEFAULT 'pending',
express_ditya(#   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
express_ditya(#   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
express_ditya(#   FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
express_ditya(# );
CREATE TABLE

```

6. Screenshot

Method	Endpoint	Success	Failed
POST	/user/register		

PUT	/user		
POST	/user/login		
POST	/user/topUp		

POST	/transaction/ create		
POST	/transaction/ pay		
DELETE	/transaction/ :id	